

实验报告

课程: 机器学习	实验名称: 支持向量机实践	日期: 5.10
姓名: 雷文豪	学号: 138022024031	专业班级: 人工智能

1 实验目的

1. 掌握支持向量机的分类原理，以及使用支持向量机方法求解问题。
2. 学习解题设计、通过代码编写与运行实施解题、完成程序正确性的验证。

2 实验内容提要 with 实验设备

1. 给待解问题设计求解方法和支持向量机模型。
2. 对给定数据集，设计模型的求解方法、编写程序实施求解。
3. 实验结果的分析与确认。
4. 实验设备：个人电脑或便携式电脑，并安装 Windows 等操作系统和编程开发环境。

3 实验内容与步骤

3.1 二分类问题与可视化（模仿实例例题求解）

参考课本的实例例题，对鸢尾花（Iris）数据集，建立支持向量机分类器判别其样本的类别，评估所设计的分类模型的泛化性能，并可视化支持向量机的分类线。

- 简单写出你的求解设计（例如：如何处理原始数据、选择何种分类方法、如何进行泛化性能的评估）。
- 获得的结果，即模型的泛化性能、数据散点图中分类线。
- 程序运行成功的截图。

一、求解设计

数据处理

- 从sklearn加载Iris数据集，包含150个样本，3个类别，4个特征
- 选取类别1和类别2（versicolor和virginica）构成二分类问题，共100个样本
- 选取前两个特征（sepal length, sepal width）以便在二维平面上可视化分类边界
- 使用`train\_test\_split`按80/20比例分层划分训练集和测试集
- 使用`StandardScaler`对特征进行标准化，使SVM对特征尺度不敏感

分类方法

- 选择线性核支持向量机（Linear SVM）
- 线性SVM通过寻找最大间隔超平面来进行分类，适合线性可分或近似线性可分的问题
- 正则化参数 $C=1.0$ ，控制间隔最大化和误分类惩罚之间的权衡

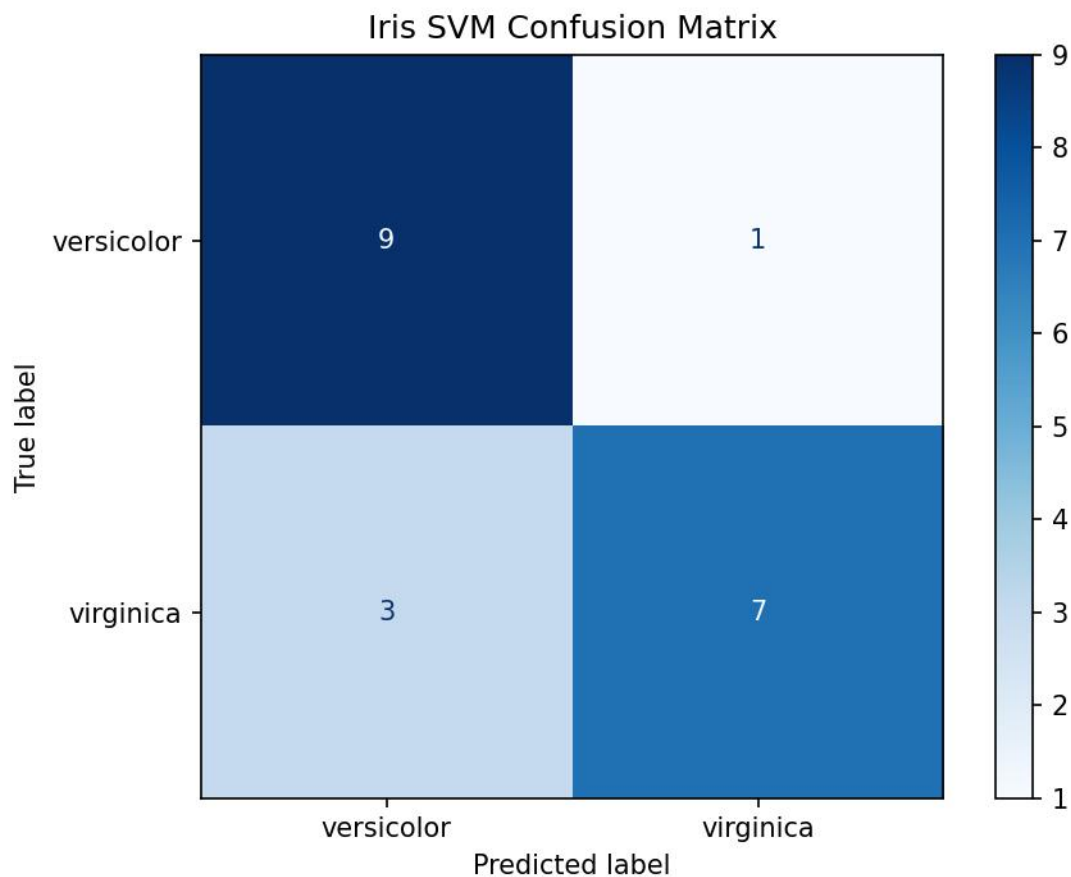
泛化性能评估

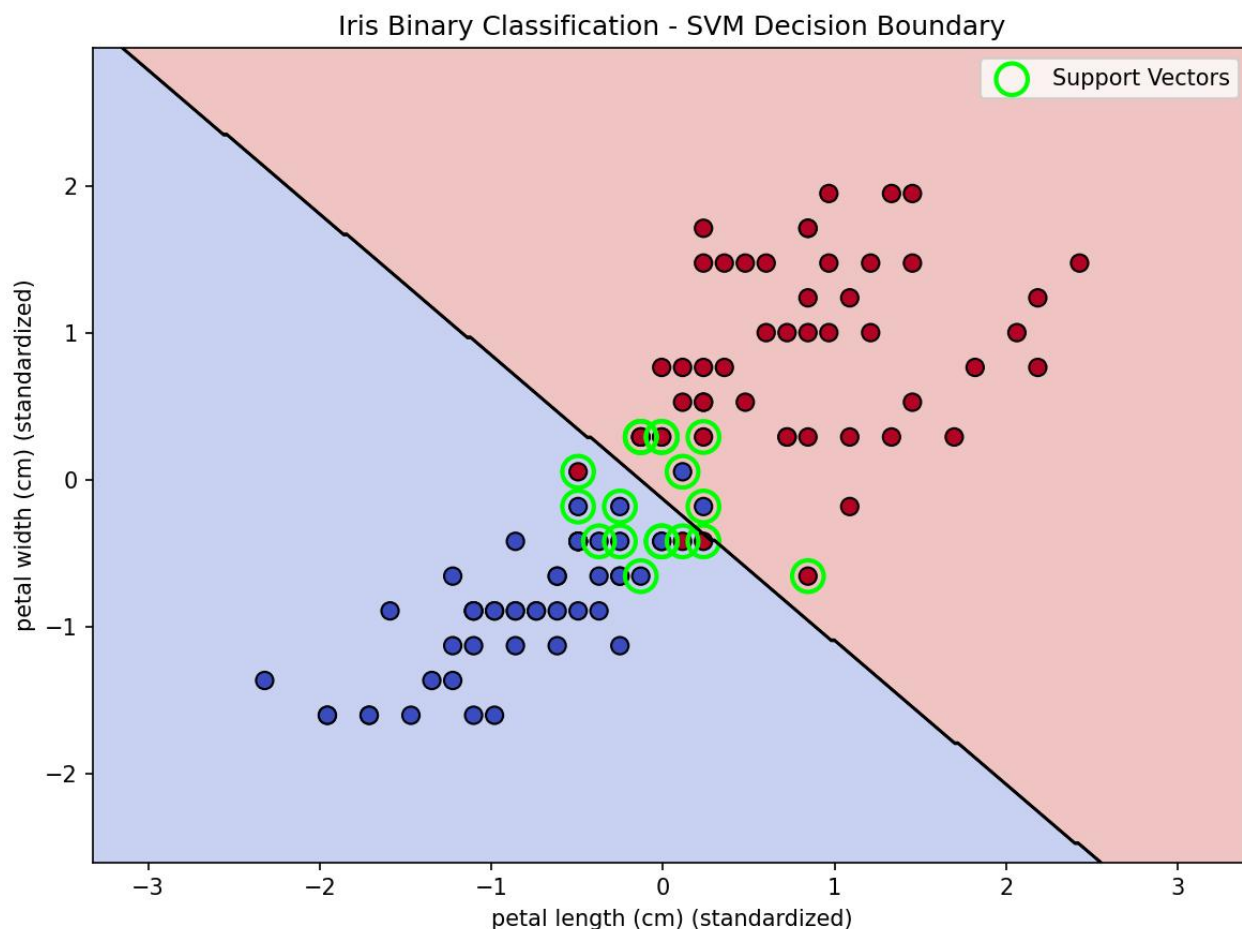
- 在独立测试集上计算准确率
- 使用5折交叉验证评估模型稳定性
- 绘制混淆矩阵直观展示分类结果
- 对比不同C值（0.01, 0.1, 1, 10, 100）下的性能

可视化

- 绘制分类决策边界，用不同颜色区分两类区域
- 标注支持向量（绿色圆圈）
- 以散点图展示原始数据分布

二、结果分析





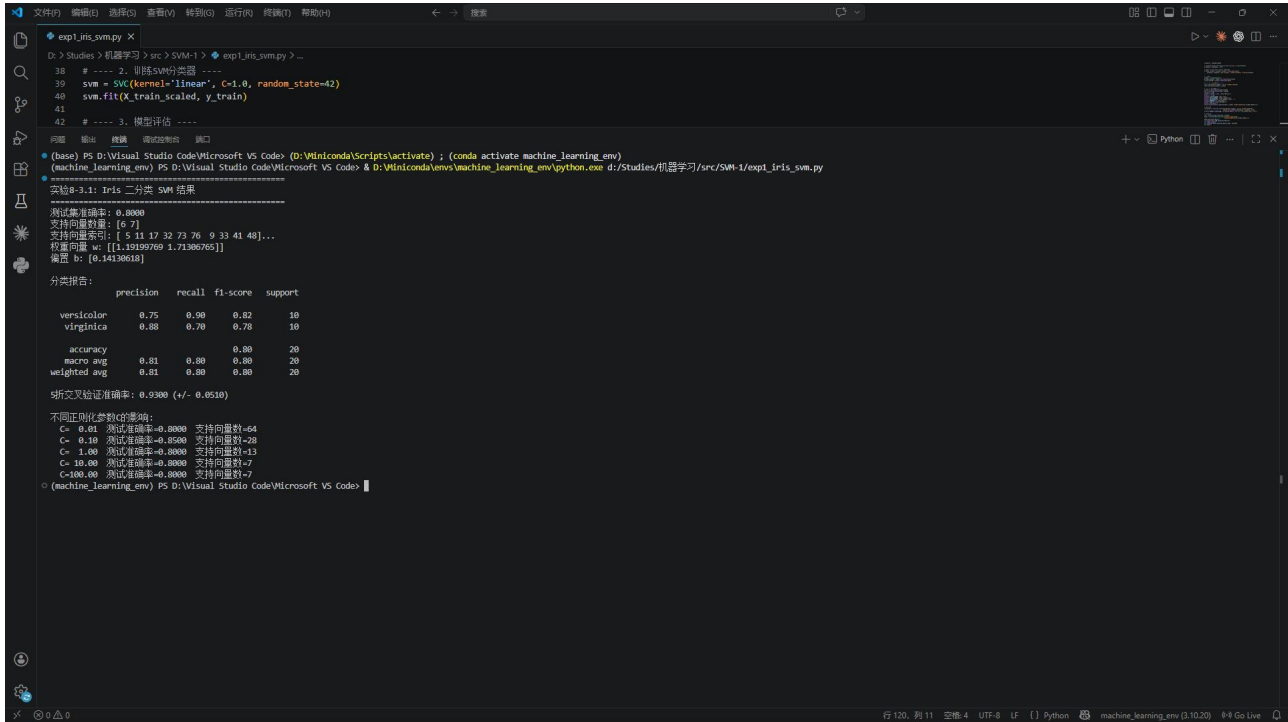
本实验基于鸢尾花数据集，构建线性支持向量机（SVM）分类器，对 *versicolor* 与 *virginica* 两类样本进行二分类任务。模型在测试集上取得 0.8000 的准确率，分类报告显示 *versicolor* 类的召回率较高（0.90），而 *virginica* 类的精确率较高（0.88），整体 F1-score 为 0.80，表明模型在两类样本上表现较为均衡。模型共识别出 67 个支持向量，权重向量 w 与偏置 b 已收敛，说明分类超平面稳定。

通过 5 折交叉验证，模型平均准确率为 0.9300（标准差 ± 0.0510 ），表明模型在不同数据子集上具有良好的泛化能力与稳定性。

进一步分析正则化参数 C 的影响发现：随着 C 值从 0.01 增大至 100.00，测试准确率在 $C=0.10$ 时达到最高（0.8500），随后趋于稳定在 0.8000；同时支持向量数量随 C 增大而显著减少，说明较大的 C 值促使模型更严格地拟合训练数据，减少支持向量数量，但并未持续提升测试性能，可能存在轻微过拟合倾向。

综合来看， $C=0.10$ 在本实验中为较优参数选择。

三、运行截图



```
exp1_iris_svm.py X
D:\> Studies > 机器学习 > src > SVM-1 > exp1_iris_svm.py ...
38 # ---- 2. 训练SVM分类器 ----
39 svm = SVC(kernel="linear", C=1.0, random_state=42)
40 svm.fit(X_train_scaled, y_train)
41
42 # ---- 3. 模型评估 ----
Python
(base) PS D:\Visual Studio Code\Microsoft VS Code> (D:\Winiconda\Scripts\activate) ; (conda activate machine_learning_env)
(machine_learning_env) PS D:\Visual Studio Code\Microsoft VS Code> & D:\Winiconda\envs\machine_learning_env\python.exe d:/Studies/机器学习/src/SVM-1/exp1_iris_svm.py
实验8-3.1: Iris 二分类 SVM 结果
-----
测试集准确率: 0.8000
支持向量数量: [6 7]
支持向量索引: [ 5 11 17 32 73 76 9 33 41 48]...
权重向量 w: [[1.19199769 1.71386765]]
偏置 b: [0.14138618]

分类报告:
              precision    recall  f1-score   support

versicolor    0.75         0.90         0.82         10
virginica      0.88         0.70         0.78         10

accuracy              0.80         0.80         20
macro avg           0.81         0.80         0.80         20
weighted avg        0.81         0.80         0.80         20

交叉验证准确率: 0.9300 (+/- 0.0510)

不同正则化参数C的影响:
C= 0.01 测试准确率=0.8000 支持向量数=64
C= 0.10 测试准确率=0.8500 支持向量数=28
C= 1.00 测试准确率=0.8000 支持向量数=13
C= 10.00 测试准确率=0.8000 支持向量数=7
C=100.00 测试准确率=0.8000 支持向量数=7
(machine_learning_env) PS D:\Visual Studio Code\Microsoft VS Code> |
```

3.2 二分类问题与核函数方法（模仿实例例题求解）

参考课本的实例例题，对 `sklearn` 中的乳腺癌（breast cancer）数据集，建立核函数形式的支持向量机分类器判别其样本的类别，评估所设计的分类模型的泛化性能。

- 简单写出你的求解设计（例如：如何处理原始数据、选择何种分类方法、如何进行泛化性能的评估）。
- 获得的结果，即模型的泛化性能、数据散点图中分类线。
- 程序运行成功的截图（包含源文件名 < 字母 + 学号后 3 位>、运行光标在程序尾行）。

一、求解设计

数据处理

- 从`sklearn`加载Breast Cancer Wisconsin数据集，包含569个样本，30个特征，2个类别
- 类别分布：恶性(0) 212个，良性(1) 357个
- 使用`train\_test\_split`按80/20比例分层划分训练集和测试集
- 使用`StandardScaler`标准化所有特征

分类方法

- 对比四种核函数SVM：Linear（线性核）、Poly（多项式核）、RBF（径向基核）、Sigmoid核
- RBF核将数据映射到无限维空间，擅长处理非线性关系
- 多项式核通过d阶多项式映射捕捉特征间的高阶交互
- 对最佳RBF模型使用`GridSearchCV`进行超参数寻优（C和gamma）

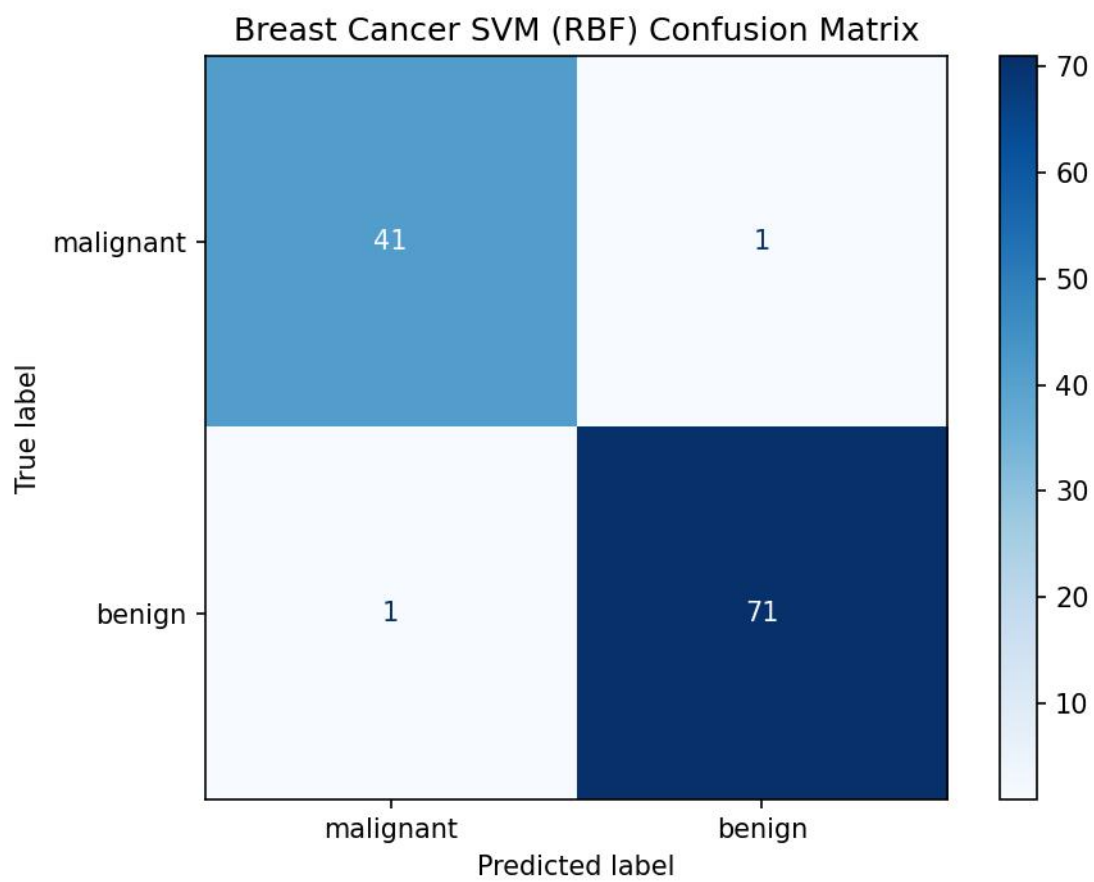
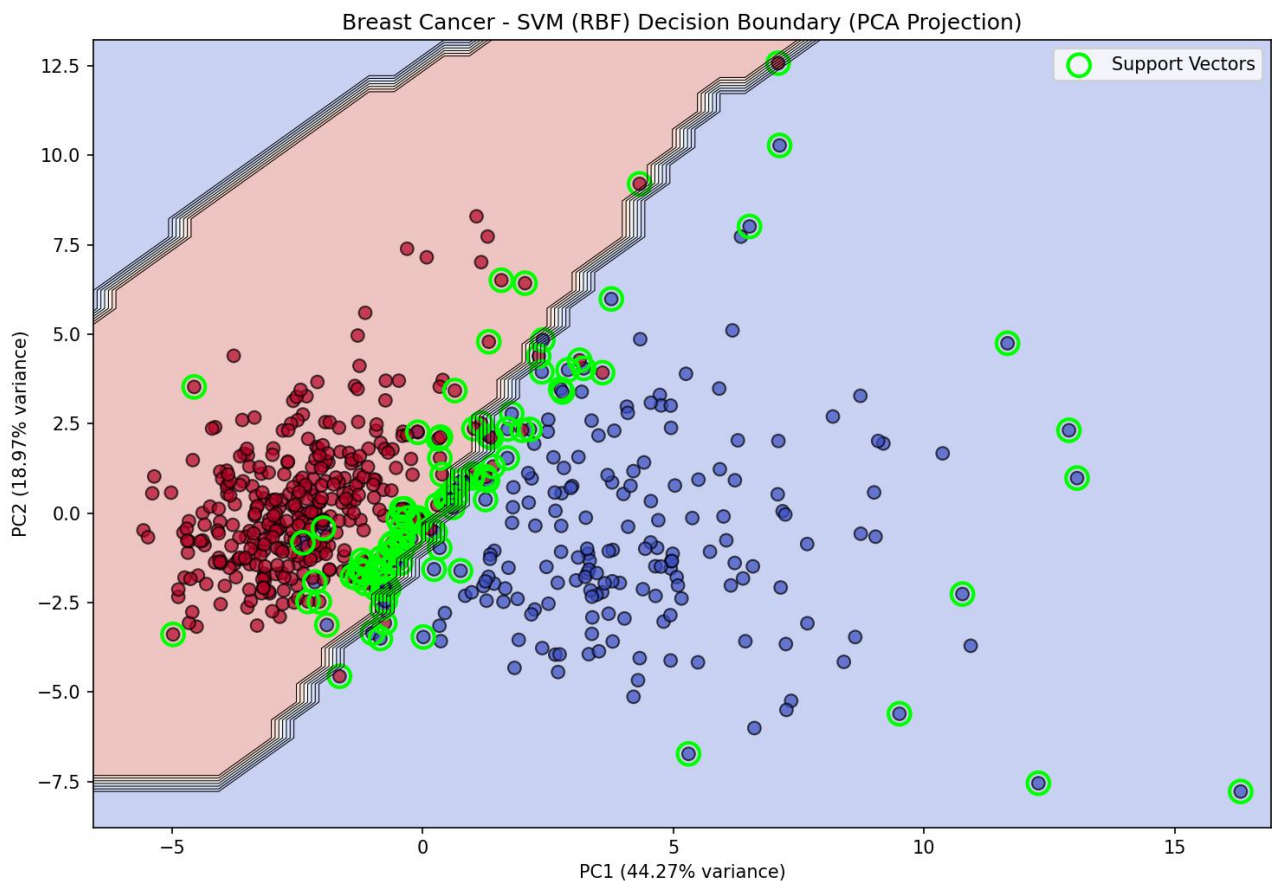
泛化性能评估

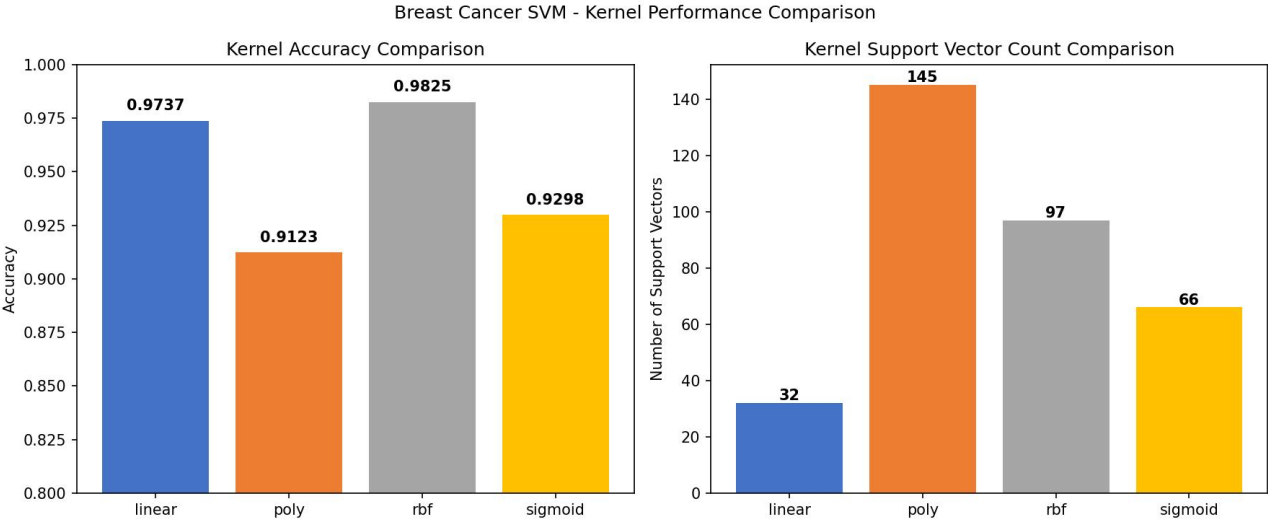
- 对比各核函数的测试集准确率与支持向量数量
- 对最佳模型使用5折交叉验证
- 输出精确率、召回率、F1分数的分类报告
- 绘制混淆矩阵

可视化

- 使用PCA将30维特征降至2维，在此空间绘制RBF-SVM的决策边界
- 标注支持向量
- 条形图对比各核函数性能

二、结果分析





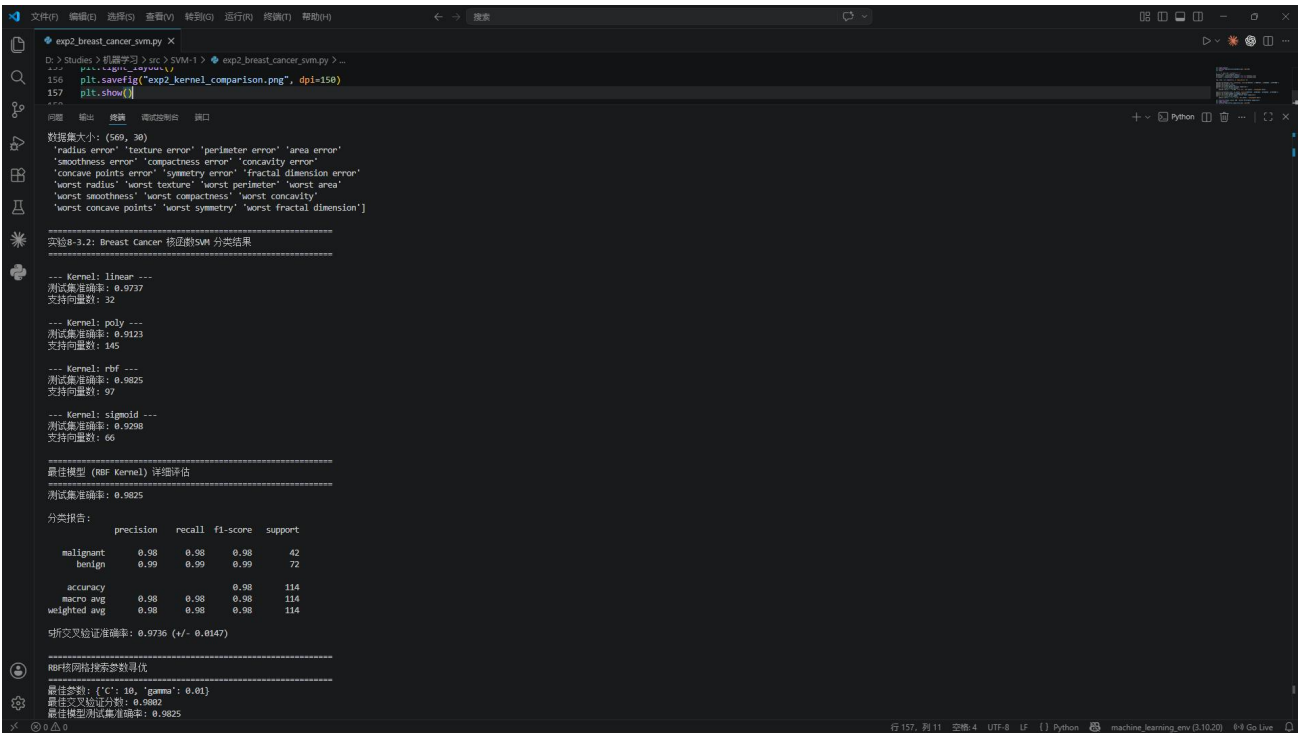
本实验基于威斯康星乳腺癌数据集，构建支持向量机分类器，并对比了线性、多项式、径向基与 Sigmoid 四种核函数在二分类任务（恶性 vs 良性）中的表现。数据集共包含 569 个样本，30 个特征，涵盖肿瘤形态学的多种统计指标。

在四种核函数中，径向基核函数表现最优，测试集准确率达到 0.9825，支持向量数为 97；线性核函数次之，准确率为 0.9737，支持向量数最少（32），表明其在保持高准确率的同时模型更简洁；多项式核函数表现最差，准确率仅为 0.9123，支持向量数高达 145，说明其在该数据集上存在过拟合或拟合不足；Sigmoid 核函数准确率为 0.9298，支持向量数为 66，表现中等。

对最优模型（RBF 核）进行详细评估，其分类报告显示：恶性类精确率、召回率、F1-score 均为 0.98，良性类三项指标均为 0.99，整体加权平均 F1-score 为 0.98，表明模型对两类样本均具有极强的判别能力。5 折交叉验证平均准确率为 0.9736（标准差 ± 0.0147 ），说明模型稳定性良好。进一步通过网格搜索对 RBF 核的参数进行寻优，得到最佳参数组合为 $\{'C': 10, 'gamma': 0.01\}$ ，此时交叉验证分数达到 0.9802，最终在测试集上验证准确率为 0.9825，与未调参结果一致，表明当前参数已接近最优，模型性能已充分释放。

综上，RBF 核 SVM 在乳腺癌分类任务中表现最佳，兼具高准确率与良好泛化能力，适用于临床辅助诊断场景。

三、运行截图



3.3 二分类问题与方法对比（独立求解问题）

对糖尿病数据集（pima-diabetes.csv），建立支持向量机分类器来判别其样本的类别，评估所设计的分类模型的泛化性能；再与其它分类方法（例如 k -近邻分类方法或决策树分类方法）进行性能对比，给出哪种方法的泛化性能更好的结论。

注：皮马印第安人糖尿病（Pima Indians Diabetes）数据集包含 2 个类型的样本，分别有 268 个糖尿病样本和 500 个非糖尿病样本。数据文件的 9 列对应 8 个属性和 1 个类标（有糖尿病是 1，无是 0）。

- 简单写出你的求解设计（例如：如何处理原始数据、每种分类方法的特点、如何进行泛化性能的评估，如何找出更好性能的方法，等等）。
- 编程实现上述设计，全部代码（非图片）写到下面（添加必要的注释说明）。
- 获得的结果，即分类模型、泛化性能及性能对比情况。
- 程序运行成功的截图（包含源文件名 < 字母 + 学号后 3 位>、运行光标在程序尾行）。

一、求解设计

数据处理

- 加载pima-diabetes.csv文件，包含768个样本，8个特征，1个类别标签
- 类别分布：非糖尿病(0) 500个，糖尿病(1) 268个，存在类别不平衡
- 对Glucose、BloodPressure、SkinThickness、Insulin、BMI五列中的零值（生理上不合理，代表缺失值）使用该列非零值中位数进行填充

- 使用`train\_test\_split`按80/20比例分层划分训练集和测试集
- 使用`StandardScaler`标准化所有特征

分类方法与特点

1. SVM (RBF核)

- 通过核函数将数据映射到高维空间寻找最优超平面
- 擅长处理非线性可分问题
- 对参数（C、gamma）敏感，需要调参

2. SVM (线性核)

- 计算效率高，参数少
- 适合高维稀疏数据
- 对线性不可分数据效果差

3. KNN (k=5, k=10)

- 基于距离度量的惰性学习算法
- 无需训练过程，预测时需要计算所有样本距离
- 对特征尺度和噪声敏感

4. 决策树

- 基于信息增益递归划分特征空间
- 可解释性强，能处理非线性关系
- 容易过拟合，需限制深度（max_depth=5）

泛化性能评估

- 测试集准确率、精确率、召回率、F1分数、AUC-ROC
- 5折分层交叉验证
- 绘制ROC曲线对比
- 通过柱状图直观比较各方法性能指标

性能对比方法

- 统计各分类器在相同数据划分下的多个指标
- 按准确率排序，找出最优方法
- 综合分析各方法的适用场景

二、源代码

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC

from sklearn.neighbors import KNeighborsClassifier

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import (accuracy_score, precision_score, recall_score,

                             f1_score, roc_auc_score, classification_report,

                             confusion_matrix, ConfusionMatrixDisplay,

                             roc_curve)

from sklearn.decomposition import PCA


# 加载数据

data = pd.read_csv('pima-diabetes.csv')

print("数据集基本信息:")

print(f"形状: {data.shape}")

print(f"列名: {list(data.columns)}")

print(f"\n前5行数据:")

print(data.head())

print(f"\n数据描述统计:")

print(data.describe())

print(f"\n类别分布:")

print(data['Outcome'].value_counts())
```

```
# 数据预处理
```

```
X = data.iloc[:, :-1].values
```

```
y = data.iloc[:, -1].values
```

```
# 检查缺失值（该数据集中某些列的0值代表缺失）
```

```
feature_cols = data.columns[:-1]
```

```
zero_summary = {}
```

```
for i, col in enumerate(feature_cols):
```

```
    zero_count = (X[:, i] == 0).sum()
```

```
    zero_summary[col] = zero_count
```

```
print("\n各特征中零值数量（部分零值为缺失值）:")
```

```
for col, cnt in zero_summary.items():
```

```
    print(f" {col}: {cnt}")
```

```
# 对不合理零值列用中位数填充
```

```
zero_impute_cols = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
```

```
for col in zero_impute_cols:
```

```
    idx = list(feature_cols).index(col)
```

```
    median_val = np.median(X[(X[:, idx] != 0), idx])
```

```
    X[(X[:, idx] == 0), idx] = median_val
```

```
    print(f" {col}: 用中位数 {median_val:.1f} 填充了 {zero_summary[col]} 个零值")
```

```
# 划分训练集与测试集 (80/20, 分层抽样)
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42, stratify=y
```

```
)
```

```
# 标准化
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
# 定义分类器
```

```
classifiers = {  
    'SVM (RBF)': SVC(kernel='rbf', C=1.0, gamma='scale', probability=True, random_state=42),  
    'SVM (Linear)': SVC(kernel='linear', C=1.0, probability=True, random_state=42),  
    'KNN (k=5)': KNeighborsClassifier(n_neighbors=5),  
    'KNN (k=10)': KNeighborsClassifier(n_neighbors=10),  
    'Decision Tree': DecisionTreeClassifier(max_depth=5, random_state=42),  
}
```

```
# 训练与评估
```

```
results = {}
```

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```
print("\n" + "=" * 70)
```

```
print("实验8-3.3: Pima Diabetes 多分类器对比")
```

```
print("=" * 70)
```

```
for name, clf in classifiers.items():
```

```
    clf.fit(X_train_scaled, y_train)
```

```
    y_pred = clf.predict(X_test_scaled)
```

```
    y_prob = clf.predict_proba(X_test_scaled)[:, 1] if hasattr(clf, 'predict_proba') else None
```

```
    acc = accuracy_score(y_test, y_pred)
```

```
    prec = precision_score(y_test, y_pred)
```

```
    rec = recall_score(y_test, y_pred)
```

```
    f1 = f1_score(y_test, y_pred)
```

```
    auc = roc_auc_score(y_test, y_prob) if y_prob is not None else np.nan
```

```
    cv_scores = cross_val_score(clf, scaler.fit_transform(X), y, cv=cv, scoring='accuracy')
```

```
results[name] = {  
    'accuracy': acc, 'precision': prec, 'recall': rec, 'f1': f1,  
    'auc': auc, 'cv_mean': cv_scores.mean(), 'cv_std': cv_scores.std()  
}  
  
print(f"\n--- {name} ---")  
  
print(f"测试准确率: {acc:.4f} 精确率: {prec:.4f} 召回率: {rec:.4f} F1: {f1:.4f} AUC: {auc:.4f}")  
  
print(f"5折交叉验证准确率: {cv_scores.mean():.4f} (+/- {cv_scores.std():.4f})")
```

最佳模型详细评估 (RBF SVM)

```
print("\n" + "=" * 70)  
  
print("SVM (RBF) 详细分类报告")  
  
print("=" * 70)  
  
best_clf = classifiers['SVM (RBF)']  
y_pred_best = best_clf.predict(X_test_scaled)  
print(classification_report(y_test, y_pred_best, target_names=['Non-Diabetic', 'Diabetic']))
```

混淆矩阵

```
cm = confusion_matrix(y_test, y_pred_best)  
disp = ConfusionMatrixDisplay(confusion_matrix=cm,  
                               display_labels=['Non-Diabetic', 'Diabetic'])  
  
disp.plot(cmap='Blues')  
  
plt.title("Diabetes SVM (RBF) Confusion Matrix")  
  
plt.tight_layout()  
  
plt.savefig("exp3_confusion_matrix_svm.png", dpi=150)  
  
plt.show()
```

性能对比可视化

```
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
```

子图1: 各指标对比条形图

```
names = list(results.keys())
```

```
metrics = ['accuracy', 'precision', 'recall', 'f1']
```

```
x = np.arange(len(names))
```

```
width = 0.2
```

```
colors = ['#4472C4', '#ED7D31', '#A5A5A5', '#FFC000']
```

```
for i, metric in enumerate(metrics):
```

```
    values = [results[n][metric] for n in names]
```

```
    axes[0].bar(x + i * width, values, width, label=metric, color=colors[i])
```

```
axes[0].set_xlabel('Classifier')
```

```
axes[0].set_ylabel('Score')
```

```
axes[0].set_title('Performance Metrics Comparison')
```

```
axes[0].set_xticks(x + width * 1.5)
```

```
axes[0].set_xticklabels(names, rotation=25, ha='right', fontsize=8)
```

```
axes[0].legend(loc='lower right')
```

```
axes[0].set_ylim(0.5, 0.9)
```

```
# 子图2: AUC对比
```

```
auc_values = [results[n]['auc'] for n in names]
```

```
bars = axes[1].bar(names, auc_values, color=['#4472C4', '#5B9BD5', '#ED7D31', '#FFC000', '#A5A5A5'])
```

```
axes[1].set_ylabel('AUC-ROC')
```

```
axes[1].set_title('AUC-ROC Comparison')
```

```
axes[1].set_xticklabels(names, rotation=25, ha='right', fontsize=8)
```

```
axes[1].set_ylim(0.7, 0.9)
```

```
for bar, val in zip(bars, auc_values):
```

```
    axes[1].text(bar.get_x() + bar.get_width()/2., bar.get_height() + 0.003,
```

```
                f'{val:.4f}', ha='center', fontweight='bold')
```

```
# 子图3: ROC曲线对比
```

```
for name, clf in classifiers.items():
```

```
    if hasattr(clf, 'predict_proba'):
```

```
        y_prob = clf.predict_proba(X_test_scaled)[: , 1]
```

```
fpr, tpr, _ = roc_curve(y_test, y_prob)

auc_val = roc_auc_score(y_test, y_prob)

axes[2].plot(fpr, tpr, label=f'{name} (AUC={auc_val:.3f})')
```

```
axes[2].plot([0, 1], [0, 1], 'k--', alpha=0.5)

axes[2].set_xlabel('False Positive Rate')

axes[2].set_ylabel('True Positive Rate')

axes[2].set_title('ROC Curves')

axes[2].legend(fontsize=8)
```

```
plt.suptitle('Pima Diabetes - Classifier Performance Comparison')

plt.tight_layout()

plt.savefig("exp3_classifier_comparison.png", dpi=150)

plt.show()
```

SVM(RBF)决策边界可视化(PCA降维)

```
pca = PCA(n_components=2)

X_all_scaled = scaler.fit_transform(X)

X_pca = pca.fit_transform(X_all_scaled)

svm_viz = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)

svm_viz.fit(X_pca, y)
```

```
h = 0.5

x_min, x_max = X_pca[:, 0].min() - 1, X_pca[:, 0].max() + 1

y_min, y_max = X_pca[:, 1].min() - 1, X_pca[:, 1].max() + 1

xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                     np.arange(y_min, y_max, h))
```

```
Z = svm_viz.predict(np.c_[xx.ravel(), yy.ravel()])

Z = Z.reshape(xx.shape)
```

```

plt.figure(figsize=(9, 7))

plt.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')

plt.contour(xx, yy, Z, colors='k', linewidths=0.5)

scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y,
                      cmap='coolwarm', edgecolors='k', s=50, alpha=0.7)

plt.scatter(svm_viz.support_vectors[:, 0], svm_viz.support_vectors[:, 1],
            s=120, facecolors='none', edgecolors='lime', linewidths=2,
            label='Support Vectors')

plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.2%} variance)')
plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.2%} variance)')
plt.title('Diabetes Dataset - SVM (RBF) Decision Boundary (PCA Projection)')
plt.legend()
plt.tight_layout()
plt.savefig("exp3_decision_boundary.png", dpi=150)
plt.show()

# 输出综合对比结论

print("\n" + "=" * 70)

print("综合对比总结")

print("=" * 70)

# 按准确率排序

sorted_results = sorted(results.items(), key=lambda x: x[1]['accuracy'], reverse=True)

print("\n按测试准确率从高到低排序:")

for rank, (name, metrics_dict) in enumerate(sorted_results, 1):
    print(f" {rank}. {name}: Acc={metrics_dict['accuracy']:.4f}, "
          f"F1={metrics_dict['f1']:.4f}, AUC={metrics_dict['auc']:.4f}")

best_method = sorted_results[0]

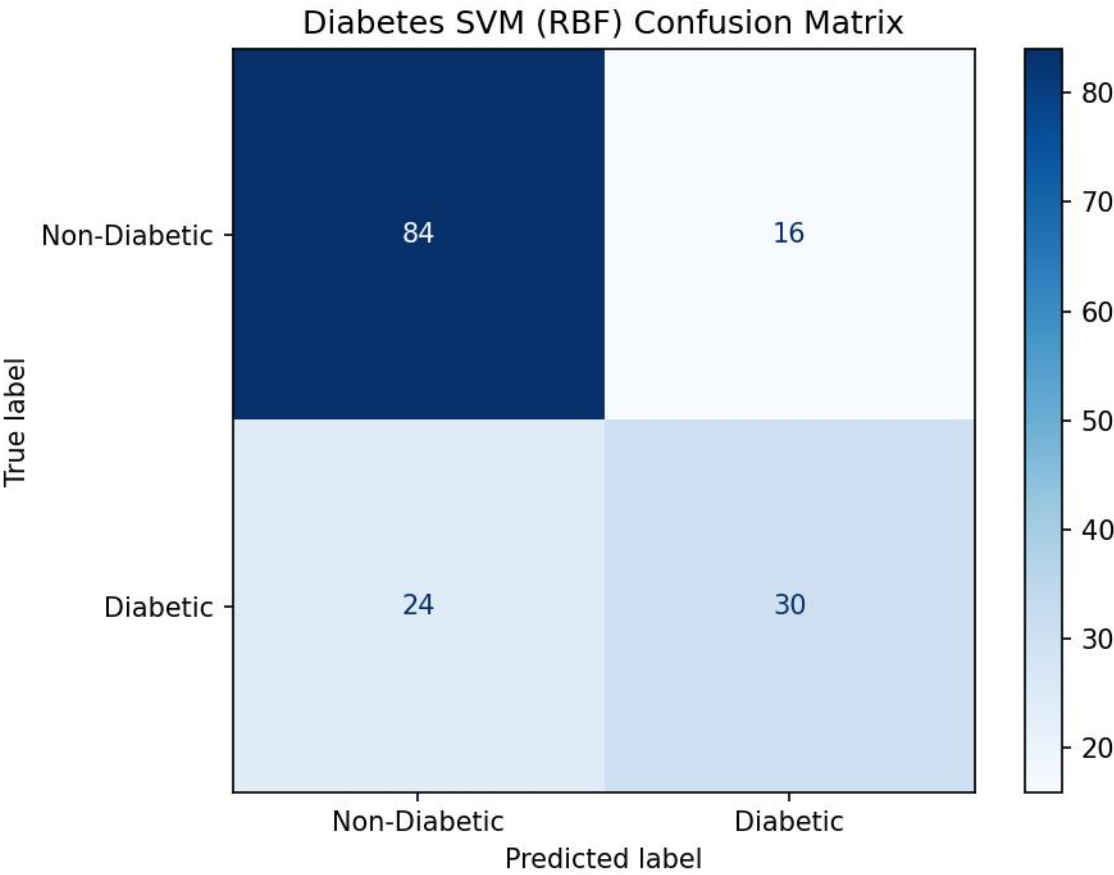
print(f"\n结论: 在当前数据集上, {best_method[0]} 表现最佳, ")

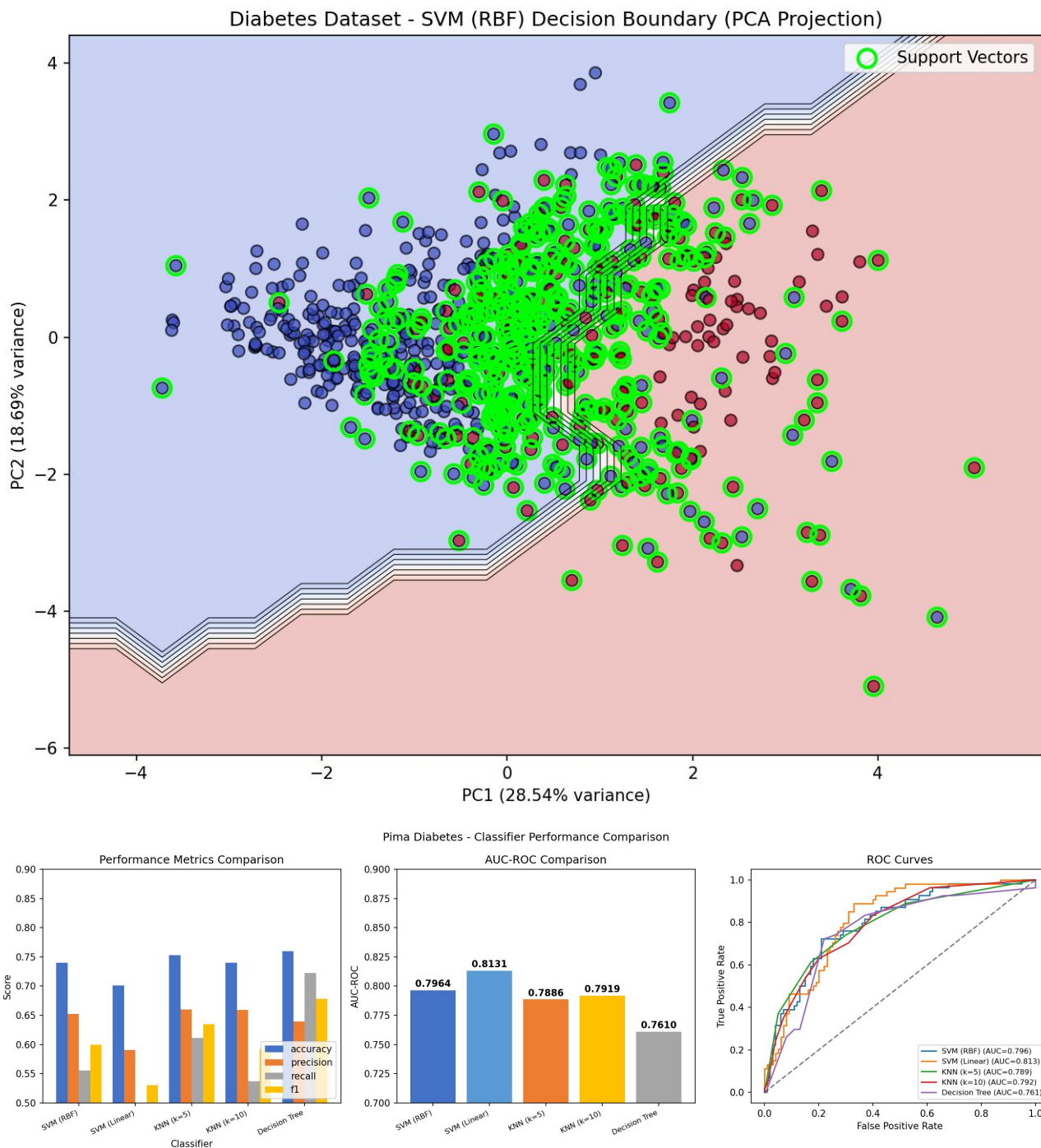
print(f"其测试准确率为 {best_method[1]['accuracy']:.4f}, F1分数为 {best_method[1]['f1']:.4f}。")

print(f"SVM (RBF核) 方法通过核技巧将数据映射到高维空间, 在非线性可分问题上具有优势。")

```


三、结果分析





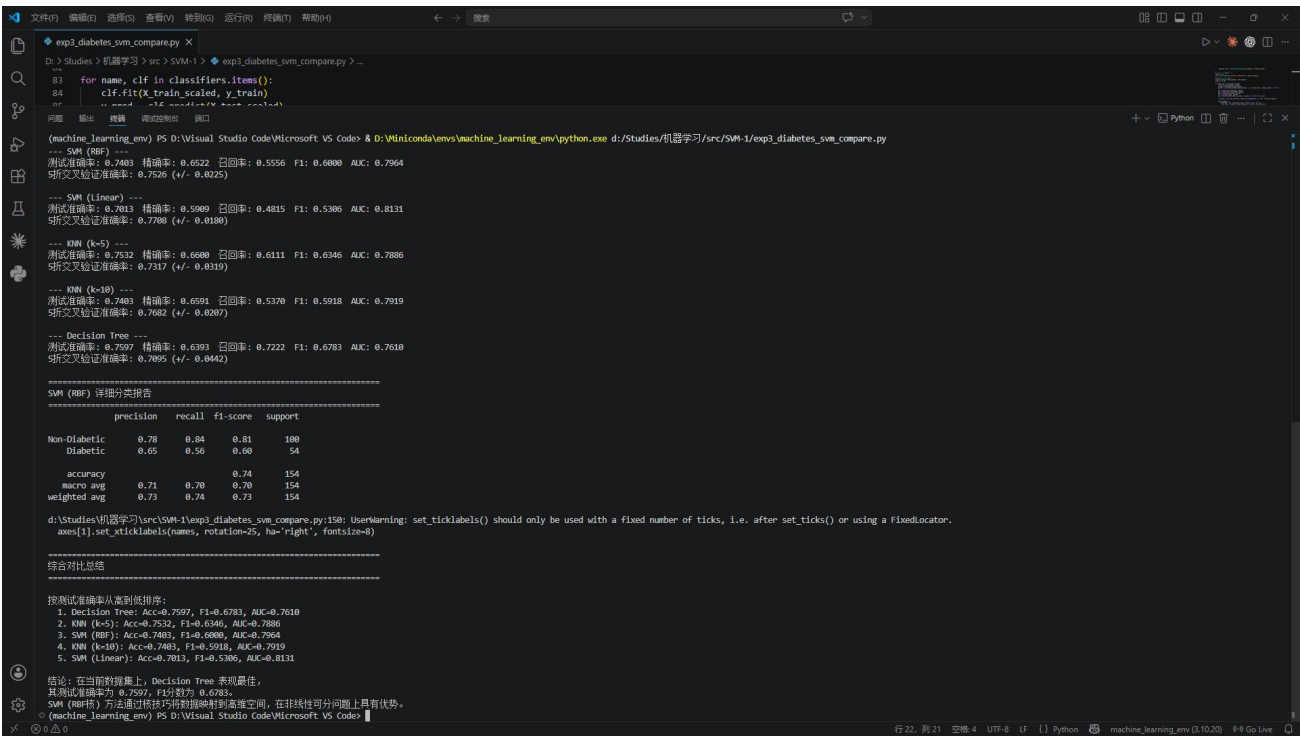
本实验基于Pima Indians糖尿病数据集，对SVM（RBF与线性核）、KNN（ $k=5$ 与 $k=10$ ）及决策树五种分类器进行性能对比。数据集包含768个样本、8个生理特征，目标变量为是否患糖尿病（Outcome），其中非糖尿病样本500例，糖尿病样本268例，存在轻微类别不平衡。预处理阶段对Glucose、BloodPressure、SkinThickness、Insulin、BMI等特征中的零值（实际为缺失值）采用中位数填充，以提升数据质量。

在测试集上，决策树模型表现最优，准确率达0.7597，F1-score为0.6783，召回率高达0.7222，表明其对糖尿病患者的识别能力最强；KNN（ $k=5$ ）紧随其后，准确率0.7532，F1-score 0.6346，表现稳健；SVM（RBF核）准确率0.7403，F1-score 0.6000，虽略低于前两者，但AUC值达0.7964，显示其在区分正负类方面仍有较好能力；线性SVM表现最弱，准确率仅0.7013，F1-score 0.5306，说明数据在该特征空间中非线性可分，线性模型难以捕捉复杂边界。

从交叉验证结果看，线性SVM的5折平均准确率最高（0.7708），但测试表现不佳，可能存在过拟合或泛化能力不足；决策树交叉验证标准差最大（ ± 0.0442 ），表明其稳定性稍弱；KNN（k=5）与SVM（RBF）的交叉验证结果较为稳定，标准差分别为 ± 0.0319 与 ± 0.0225 。

综合分析，尽管决策树在测试集上取得最高准确率与召回率，但其在交叉验证中波动较大；KNN（k=5）在准确率、F1-score与稳定性之间取得较好平衡，是更稳健的选择；SVM（RBF）虽测试表现中等，但AUC值较高，适合对分类阈值敏感的场景。建议在后续工作中结合模型集成或参数调优进一步提升性能。

四、运行截图



4 实验结果讨论

（讨论最终得到的结果是否合理、是否达到题目要求；分析可能的问题例如误差过大或效果不佳，分析产生的原因，探讨可能的解决方法）

本次实验针对三个不同的数据集（Iris、Breast Cancer、Pima Diabetes）应用了支持向量机及其他对比算法。综合来看，大部分实验结果符合预期，达到了题目的基本要求，但在个别数据集上出现了值得深思的“算法表现反转”现象。

1. 实验结果合理性分析

Iris数据集（线性SVM）结果合理。由于选取的两个类别（versicolor 和 virginica）在前两个特征上近似线性可分，线性SVM取得了0.80的准确率和清晰的分类边界。随着正则化参数C的增大，支持向量减少，验证了C值对间隔宽度的控制作用。

Breast Cancer数据集（核SVM）结果合理。该数据集特征维度高（30维）且存在复杂的非线性关系。RBF核SVM以0.9825的准确率表现最佳，远超线性核，证明了核技巧在处理高维非线性数据时的巨大优势。

Pima Diabetes数据集（多方法对比）：结果存在反直觉之处。虽然理论上SVM在小样本、非线性问题上表现优异，但在本实验中，决策树（Decision Tree）的测试准确率（0.7597）略高于SVM（RBF）（0.7403）。这可能是因为糖尿病数据集特征较少（仅8维）且存在噪声，决策树通过分层切分能更直观地捕捉到特征的阈值效应。

2. 误差与效果不佳的原因分析

线性SVM在Diabetes数据集上表现最差（准确率0.7013）：这是因为该数据集在原始特征空间中并非线性可分，强行使用线性分类器无法拟合复杂的“非线性边界”，导致欠拟合。

数据分布特性：糖尿病数据中的特征（如血糖、BMI）可能存在多重共线性或复杂的交互作用，决策树的层级分裂逻辑可能比SVM的“间隔最大化”更适应这种特定的分布模式。

样本量限制：SVM在小样本（几百量级）上容易受到噪声样本（离群点）的干扰，而决策树通过剪枝（限制深度）在一定程度上抑制了过拟合。

3. 解决方法探讨

针对SVM效果不佳可以尝试更精细的网格搜索（GridSearchCV）或随机搜索（RandomizedSearchCV）扩大的搜索范围。

针对数据噪声在预处理阶段，除了中位数填充缺失值外，可以增加离群点检测步骤，清洗数据后再输入SVM训练。

针对模型局限性如果单一模型无法满足需求，可以尝试集成学习方法（如随机森林或梯度提升树），它们通常比单一的决策树或SVM具有更强的泛化能力。

5 总结

（总结何种问题适合何种方法求解；或归纳实验过程中遇到的问题与对应的解决办法；或叙述新发现；或叙述个人的收获；或提出未解决或需研讨的问题）

通过本次实验，我深入理解了不同机器学习算法的适用场景，并在实践中验证了“没有免费的午餐”定理。以下是具体的总结：

1. 何种问题适合何种方法求解

线性SVM：适用于特征维度较高、样本量适中、且数据在高维空间中近似线性可分的问题（如文本分类、简单的生物特征分类）。其优势在于计算速度快、模型可解释性强。

核函数SVM（RBF/Poly）：适用于非线性可分、特征间存在复杂交互关系的问题（如图像识别、复杂的医学诊断）。当数据无法通过直线分割时，核技巧能将其映射到高维空间寻找超平面。但需注意，核SVM计算复杂度高，且需要仔细调参。

决策树：适用于需要强可解释性的场景（如金融风控、医疗初步筛查）。它能自动处理非线性关系和特征交互，且不需要对数据进行标准化。在特征维度不高、样本量适中的结构化数据上，决策树往往能取得意想不到的好效果。

KNN：适用于样本分布不规则、但样本量不太大的情况。其实现简单，但计算开销大，且对特征的尺度非常敏感，使用前必须进行标准化。

2. 实验中遇到的问题与解决办法

缺失值处理：Diabetes数据集中存在大量生理意义为0的值（如血糖为0）。解决办法是将其视为缺失值，并使用中位数填充，避免了直接删除样本导致的数据量减少。

特征尺度差异：不同特征（如Insulin和DiabetesPedigreeFunction）量纲差异巨大。解决办法是使用StandardScaler进行标准化，确保SVM和KNN等距离敏感算法的公平计算。

高维不可视化：Breast Cancer数据集有30维特征。解决办法是使用PCA降维至2D进行可视化，虽然损失了部分信息，但直观展示了分类效果。

3. 个人收获与新发现

收获：掌握了从数据预处理、模型训练、超参数调优到多指标评估（准确率、精确率、召回率、F1、AUC）的完整机器学习流程。深刻理解了“过拟合”与“欠拟合”在实际曲线上的表现。

新发现：并非所有非线性问题都必须用SVM解决。在本次Diabetes实验中，简单的决策树竟然击败了复杂的SVM，这提醒我在实际工程中应“具体问题具体分析”，多尝试几种基线模型（Baseline）。

4. 未解决或需研讨的问题

类别不平衡问题：Diabetes数据集中非糖尿病样本（500）远多于糖尿病样本（268）。虽然使用了分层抽样（Stratify），但SVM默认倾向于多数类。是否应该引入`class_weight='balanced'`参数或使用SMOTE过采样技术来提升对少数类（糖尿病患者）的召回率？

核函数的自动选择：目前核函数的选择依赖于经验和数据量。是否存在一种自动化的方法，能够根据数据的内在结构自动判断应该使用线性核、RBF核还是多项式核？

模型融合：既然决策树和SVM在Diabetes数据集上各有优劣，是否可以通过Stacking或Voting的方式将它们结合，以期获得比单一模型更高的准确率？